

BtcLoan

Business Requirements Document

Bitcoin-backed micro-loan engine for microfinance institutions

Prepared for: Tobe (project owner)

Prepared by: Draft compiled from working design discussion

Version: 0.2 — draft for internal review

Date: September 15, 2026

Status: Pre-build — requirements baseline ahead of technical design and implementation

This document defines the business and functional requirements for BtcLoan, a Bitcoin-backed micro-loan engine built as an open source core with closed source, merchant-specific extensions. It exists to align on scope and the end-to-end borrower and operator flow before technical design and implementation begin.

Table of contents

Table of contents	2
1. Document control	4
2. Executive summary	4
3. Business objectives	4
4. Scope	5
4.1 In scope — open source core	5
4.2 In scope — closed source merchant extensions	5
4.3 Out of scope for this document	5
5. System context	5
6. Actors and roles	6
7. End-to-end flow: authentication to finalization	6
7.1 Authentication and identity	6
7.2 Loan product selection and application	6
7.3 Underwriting and credit scoring	6
7.4 Collateral deposit and custody	6
7.5 Approval and disbursement	6
7.6 Active loan monitoring	7
7.7 Margin call handling	7
7.8 Liquidation	7
7.9 Repayment processing	7
7.10 Finalization and closure	7
7.11 Post-loan reporting	7
8. Functional requirements	8
8.1 Authentication and identity	8
8.2 Loan product and application	8
8.3 Underwriting and scoring	8
8.4 Collateral and custody	9
8.5 Approval and disbursement	9
8.6 Monitoring, margin calls, and liquidation	9

8.7 Repayment and finalization 10

8.8 Configuration and multi-tenancy 10

9. Non-functional requirements 11

10. Core data entities 11

11. Assumptions and constraints 11

12. Open decisions 12

13. Glossary 12

1. Document control

Field	Value
Document title	BtcLoan — Business Requirements Document
Version	0.2 (draft)
Status	Pre-build baseline, open for review before technical design
Related architecture note	Open core / SDK / closed merchant extension model (see Section 5)
Next review trigger	Any change to loan lifecycle scope, or first merchant extension kickoff
v0.2 change summary	Custody generalized from BTCPay-specific to a pluggable custody provider interface (BTCPay as default); credit scoring reframed as an aggregator over the core's own score plus optional external signal providers

2. Executive summary

BtcLoan is a Bitcoin-backed micro-loan engine intended for microfinance institutions (MFIs) and similar lenders. Borrowers pledge Bitcoin as collateral, held through a pluggable custody provider (BTCPay Server is the default, open source implementation, though the core is not bound to it), against a fiat or stablecoin loan. The system continuously monitors loan-to-value (LTV), issues margin calls as the Bitcoin price moves, and liquidates collateral automatically if a borrower fails to respond, while the MFI itself remains the lender of record.

The system is split into an open source core, covering generic loan lifecycle, custody, LTV monitoring, and a base credit scoring model, and closed source merchant extensions that carry institution-specific underwriting rules, integrations, and regulatory reporting. Every behavioural parameter, from LTV thresholds to margin call timing to loan product terms, is intended to be configuration rather than code, so a single core deployment can serve multiple merchants with materially different rules.

This document exists to fix the end-to-end flow, from a borrower authenticating through to a loan reaching finalization, and the functional and non-functional requirements that flow support implies, before any code is written.

3. Business objectives

ID	Objective
BO-1	Let an MFI offer Bitcoin-collateralized micro-loans without building custody, LTV monitoring, or liquidation infrastructure themselves.
BO-2	Keep the MFI as lender of record at all times; BtcLoan is infrastructure, not a lending license substitute.
BO-3	Make every lending parameter configurable per merchant and per loan product, with no merchant-specific logic hardcoded into the core.
BO-4	Allow the generic engine to be self-hosted for free as open source, while merchant-specific underwriting, integrations, and compliance are delivered as paid, closed source extensions.

ID	Objective
BO-5	Support extensions being built by the project owner directly, by the project owner under a paid merchant engagement, or independently by third-party developers against a published SDK.
BO-6	Protect borrowers and MFIs from silent liquidation or missed margin calls through predictable, auditable, event-driven monitoring.

4. Scope

4.1 In scope — open source core

Loan lifecycle state machine; Bitcoin collateral custody through a pluggable custody provider interface, with BTCPay Server shipped as the default open source implementation; continuous LTV monitoring against a configurable price oracle; margin call and liquidation logic; a credit scoring aggregator with the core's own Bitcoin credit score as the default, primary signal; the tenant and product configuration engine; and the event bus and provider interfaces that extensions build against.

4.2 In scope — closed source merchant extensions

Merchant-specific underwriting rules; integration with a merchant's KYC provider and core banking or ledger system; regulatory and compliance reporting for the merchant's jurisdiction; white labelling and merchant-branded borrower experience; and any business rule that is specific to one institution rather than generically useful.

4.3 Out of scope for this document

Specific technology choices, database schema, API contracts, and UI design are technical design decisions that follow this BRD rather than being fixed by it. Licensing terms for the core and SDK are noted as an open decision (Section 12) rather than finalized here.

5. System context

Three layers make up the system. The open source core is the generic engine, deployable standalone by any lender. The SDK, also open source, is the published contract of interfaces and events that extensions build against without ever forking core. Closed source merchant extensions implement that contract to add institution-specific behaviour, and are built by the vendor, by the vendor on a merchant's behalf, or by independent third parties.

Layer	License	Responsibility
Core	Open source	Loan engine, custody interface (BTCPay default), LTV/margin/liquidation, scoring aggregator, config engine
SDK	Open source	Interfaces and events extensions implement; versioned and stability-graded
Extensions	Closed source	Merchant-specific underwriting, integrations, compliance, branding

6. Actors and roles

Role	Description
Borrower	Individual or small business pledging Bitcoin collateral for a loan.
Loan officer	Merchant staff who reviews applications flagged for manual underwriting and approves disbursement.
Compliance officer	Merchant staff responsible for KYC/AML sign-off and regulatory reporting.
Merchant administrator	Configures loan products, LTV and margin call rules, and staff access for their tenant.
System administrator	Operates the core deployment; manages tenants, core-level configuration, and platform health.
Automated system	The core engine itself acting on events: LTV monitor, margin call issuer, liquidation executor.

7. End-to-end flow: authentication to finalization

This is the backbone flow the system must support. Each stage below is followed by the functional requirements it implies in Section 8, and notes which layer, core or extension, is expected to own the behaviour.

7.1 Authentication and identity

A borrower registers or signs in through a merchant-branded portal. Merchant staff (loan officers, compliance, administrators) authenticate separately with role-based permissions scoped to their tenant. KYC and AML verification runs against whichever provider the merchant's extension integrates, against a generic KYC provider interface the core defines.

7.2 Loan product selection and application

The borrower browses loan products configured for that tenant (LTV limits, tenor, interest structure, supported currencies) and submits an application specifying requested amount and intended collateral.

7.3 Underwriting and credit scoring

The core's generic Bitcoin credit score is computed from on-chain and repayment history. A merchant extension may layer additional, institution-specific underwriting signals. The outcome is auto-approval below a configured risk threshold, auto-rejection above another, or routing to a loan officer's manual review queue.

7.4 Collateral deposit and custody

The borrower sends Bitcoin to a deposit address generated through the tenant's configured custody provider, BTCPay Server by default, though any implementation of the core's custody interface can be substituted. The core waits for the configured confirmation depth, records the deposit, and computes the initial LTV against the requested loan amount.

7.5 Approval and disbursement

Once collateral is confirmed and underwriting has cleared, the loan is approved automatically or by a loan officer depending on configuration, and disbursed in the tenant's configured settlement currency. The loan enters Active state and an audit log entry and borrower notification are generated.

7.6 Active loan monitoring

While a loan is active, the core continuously recomputes LTV against a live Bitcoin price feed. Threshold breaches raise an LTV breach event on the core's event bus for any subscriber, core or extension, to act on.

7.7 Margin call handling

Configured LTV tiers trigger margin calls with a response window. The borrower is notified through whichever channel the merchant extension implements, which matters in markets with mixed smartphone penetration, and can top up collateral or make a partial repayment to cure the call.

7.8 Liquidation

If a margin call is not cured within its window, the core executes liquidation of sufficient collateral to restore the configured LTV, settles the outstanding balance, and returns any remaining collateral to the borrower.

7.9 Repayment processing

Scheduled and ad hoc repayments are tracked against the loan's amortization schedule. Partial repayment may release a proportional share of collateral if the tenant's configuration allows it.

7.10 Finalization and closure

Full repayment releases all remaining collateral to the borrower and moves the loan to Closed state. The borrower's credit score updates based on repayment performance, and a final statement is generated for the MFI's own reconciliation.

7.11 Post-loan reporting

Closed and liquidated loans feed into regulatory reporting, which is extension-specific per jurisdiction, and into the core's generic audit trail, which is retained regardless of extension.

8. Functional requirements

Requirements are grouped by the flow stage they support. The Layer column indicates whether the requirement is expected to live in the open source core or be implemented per merchant as an extension.

8.1 Authentication and identity

ID	Requirement	Priority	Layer
FR-1.1	Borrowers can register and authenticate through a merchant-branded interface.	Must	Extension
FR-1.2	Merchant staff authenticate with role-based access scoped to their own tenant only.	Must	Core
FR-1.3	The core exposes a generic KYC provider interface; extensions implement it against a specific vendor.	Must	Core / Ext.
FR-1.4	A borrower cannot proceed past KYC verification to loan application while unverified.	Must	Core
FR-1.5	Session and credential handling supports revocation and forced re-authentication for compliance holds.	Should	Core

8.2 Loan product and application

ID	Requirement	Priority	Layer
FR-2.1	Merchant administrators can define loan products with tenant-specific LTV limits, tenor, currency, and interest terms.	Must	Core (config)
FR-2.2	Borrowers can view only the products configured as active for their tenant.	Must	Core
FR-2.3	An application captures requested amount, intended collateral, and selected product.	Must	Core

8.3 Underwriting and scoring

ID	Requirement	Priority	Layer
FR-3.1	The core computes a base Bitcoin credit score from on-chain and repayment history; this is the primary, always-available signal.	Must	Core
FR-3.2	Scoring is an aggregator: the core's score combines with zero or more additional weighted signal providers (merchant alternative data, KYC risk signal, future external reputation sources) behind one scoring interface.	Must	Core
FR-3.3	Extensions can register additional scoring signal providers without altering the core's own scoring logic.	Must	Ext.
FR-3.4	Applications route to auto-approve, auto-reject, or manual review based on configured risk thresholds.	Must	Core (config)

ID	Requirement	Priority	Layer
FR-3.5	A loan officer can view the combined score and the contribution of each signal behind a manual review routing.	Should	Core

8.4 Collateral and custody

ID	Requirement	Priority	Layer
FR-4.1	The core defines a custody provider interface (address generation, deposit detection, balance query, release, liquidation transfer); the loan engine calls only this interface, never a specific custodian.	Must	Core
FR-4.2	BTCPay Server ships as the default, open source implementation of the custody provider interface.	Must	Core
FR-4.3	A tenant can configure an alternate custody implementation (multisig/timelock, third-party custodian, exchange custody) in place of the default, without any core code change.	Must	Core (config) / Ext.
FR-4.4	Collateral is recognised only after a configurable confirmation depth is reached.	Must	Core (config)
FR-4.5	Initial LTV is computed and stored at the moment collateral is confirmed.	Must	Core

8.5 Approval and disbursement

ID	Requirement	Priority	Layer
FR-5.1	Disbursement currency and channel are configurable per tenant and per product.	Must	Core (config)
FR-5.2	Loans below a configured risk and amount threshold can disburse without manual approval.	Should	Core (config)
FR-5.3	Every disbursement writes an immutable audit log entry.	Must	Core
FR-5.4	The borrower is notified on disbursement through the merchant's configured notification channel.	Must	Extension

8.6 Monitoring, margin calls, and liquidation

ID	Requirement	Priority	Layer
FR-6.1	LTV recalculates continuously against a configurable Bitcoin price oracle.	Must	Core
FR-6.2	An LTV breach beyond a configured tier emits a margin call with a configurable response window.	Must	Core (config)
FR-6.3	A borrower can cure a margin call by topping up collateral or making a partial repayment.	Must	Core
FR-6.4	Failure to cure within the window triggers automatic liquidation of sufficient collateral.	Must	Core

ID	Requirement	Priority	Layer
FR-6.5	Liquidation execution and outcome are recorded on the event bus for any extension to act on.	Must	Core
FR-6.6	Margin call notification channel (push, SMS, USSD, email) is merchant-configurable.	Must	Extension

8.7 Repayment and finalization

ID	Requirement	Priority	Layer
FR-7.1	Repayments are tracked against a configurable amortization schedule.	Must	Core
FR-7.2	Partial repayment can trigger proportional collateral release if enabled for the product.	Should	Core (config)
FR-7.3	Full repayment releases remaining collateral and closes the loan.	Must	Core
FR-7.4	Credit score updates based on repayment performance on closure or liquidation.	Must	Core
FR-7.5	A closed or liquidated loan generates a merchant-facing statement for reconciliation.	Must	Extension

8.8 Configuration and multi-tenancy

ID	Requirement	Priority	Layer
FR-8.1	Every behavioural parameter in this document is expressed as configuration, not code.	Must	Core
FR-8.2	Configuration resolves at system, tenant, and product level, with product overriding tenant overriding system.	Must	Core
FR-8.3	A tenant's data, credentials, and configuration are isolated from every other tenant.	Must	Core

9. Non-functional requirements

Category	Requirement
Security	Custody keys and credentials are never exposed to extension code directly; extensions interact only through the core's provider interfaces.
Auditability	Every state transition (application, approval, deposit, margin call, liquidation, repayment, closure) is logged immutably.
Configurability	No merchant-specific rule may require a core code change to implement; if it does, that is a signal the SDK needs a new extension point.
Availability	LTV monitoring and margin call issuance are near-real-time; a delay here has direct financial consequences for borrower and MFI.
Compatibility	Extensions built against a given SDK version continue to function across core patch and minor releases within that SDK's supported range.
Data residency	Tenant configuration supports data handling appropriate to the merchant's regulatory jurisdiction.
Localization	Borrower-facing notification channels and language are merchant-configurable, not hardcoded to one market.

10. Core data entities

Entity	Purpose
Tenant	A merchant/MFI using the system; owns configuration, staff, and its own loan book.
LoanProduct	A configured product template: LTV limits, tenor, currency, interest structure.
Borrower	The individual or business applying for and holding loans.
LoanApplication	A submitted request, prior to approval, carrying underwriting outcome.
Loan	An approved, disbursed loan and its current lifecycle state.
CollateralDeposit	A recorded Bitcoin deposit against a specific loan, with confirmation status.
LTVEvent	A point-in-time LTV calculation, used for monitoring and history.
MarginCall	An issued margin call, its tier, window, and cure status.
Liquidation	A liquidation execution record, amount liquidated, and resulting balances.
Repayment	A recorded repayment, scheduled or ad hoc, against a loan.
CreditScore	The borrower's current and historical Bitcoin credit score.
AuditLog	An immutable record of state-changing actions across the system.

11. Assumptions and constraints

ID	Assumption / constraint
A-1	The MFI remains the licensed lender of record; BtcLoan does not itself hold a lending license.
A-2	BTCPay Server is the default custody implementation, chosen for day-one delivery, but the custody interface itself is a first-class extension point from the start, not a later refactor.
A-3	A live, reasonably reliable Bitcoin price feed is available to the deployment; feed reliability is itself a dependency worth stress-testing before launch.
A-4	Regulatory reporting requirements vary enough by jurisdiction that they are assumed to always live in the extension layer, never in core.
A-5	The first merchant extension is expected to be built by the project owner, ahead of publishing the SDK for third-party use.
A-6	No mature, widely adopted, verifiably open source Bitcoin-native credit scoring standard exists at the time of writing; the core's own scoring engine is assumed to be the primary signal for the foreseeable term, with the aggregator interface left open for external sources as the space matures.

12. Open decisions

These are flagged rather than resolved here, since they affect technical design but are business decisions first.

ID	Decision needed
OD-1	Final license choice for core and SDK (for example MIT versus Apache 2.0 with restrictions on competing hosted forks).
OD-2	Which margin call and liquidation parameters ship as sensible platform defaults versus requiring explicit merchant configuration before a product can go live.
OD-3	Whether proportional collateral release on partial repayment is a core default or an opt-in per product.
OD-4	Scope of the first merchant extension's KYC and core banking integrations, which will shape how the KYC and ledger provider interfaces are first drafted.

13. Glossary

Term	Meaning
LTV	Loan-to-value: outstanding loan balance divided by current collateral value.
Margin call	A notice requiring the borrower to add collateral or repay to bring LTV back within limits.
Liquidation	Selling or converting enough pledged collateral to restore the loan to a safe LTV.
MFI	Microfinance institution; the licensed lender using BtcLoan as infrastructure.
Core	The open source generic engine, deployable standalone.
Extension	A closed source, merchant-specific package built against the core's SDK.
Tenant	A single merchant's isolated configuration, data, and loan book within a deployment.

End of document. Next step: technical design derived from Section 8, followed by SDK interface drafts for the provider and event contracts referenced throughout.